Id1: 345020226, name1: Saba Golbandi, user1: sabagolbandi
Id2: 325394534, name2: Nasma Shannan, user2: nasmashannan

Experiments

Part 1

עלות איזון במערך עם היפוכים סמוכים אקראיים	עלות איזון במערך מסודר אקראית	עלות איזון במערך ממוין-הפוך	עלות איזון במערך ממוין	i מספר סידורי
417.3	384.1	430	430	1
843.35	782.3	873	873	2
1689.5	1571.4	1760	1760	3
3372.7	3159.05	3535	3535	4
6792.2	6307.5	7086	7086	5
13623.05	12669.9	14189	14189	6
27278.5	25353.95	28396	28396	7
54496.75	50755.3	56811	56811	8
109221.45	101447.3	113642	113642	9
218247.85	202826.0	227305	227305	10

Since in each insert, we check the path from the insertion point up until the root, the asymptotic number of promotions done in each insert is bounded by $\log n$, where n is the number of the items in the tree. So, the total upper bound will be:

$$\log 1 + \log 2 + \cdots + \log n = \sum_{i=0}^{n} \log i = \log(n!) = O(n \log n)$$

However, since we are dealing with a sequence of insertions, we can consider the total cost as $O(n)$, like what we saw in the lecture, and then the amortized upper bound will be $O(n)$.
Now, in our case of $n = 111 \cdot 2^i$, for each i, the upper bound of total cost will be
$n \log n = 111 \cdot 2^i \cdot \log(111 \cdot 2^i) = 111 \cdot 2^i \cdot (\log 111 + i) \leq 111 \cdot 2^i (7 + i)$.
That is as the following table for each i:

i	The upper bound $O(n \log n)$	The amortized upper bound $O(n)$
1	1776	222
2	3996	444
3	8880	888
4	19536	1776
5	42624	3552
6	92352	7104
7	198912	14208
8	426420	28416
9	909312	56832
10	1932288	113664

Hence, as we can see from the experiments table, all the costs resulted from the experiments are less than these upper bounds, and in practice, the real asymptotic bound is the amortized bound.
Furthermore, in the worst case, the sum of promotions and rotations will not be greater than the height of the tree, $\log n$, so the total cost in this case will be bounded by the upper bound as well. Hence, the cost of the rotations does not affect the total cost asymptotically.

Part 2

מספר היפוכים במערך עם היפוכים סמוכים אקראיים	מספר היפוכים במערך מסודר אקראית	מספר היפוכים במערך ממוין-הפוך	מספר היפוכים במערך ממוין	מספר סידורי i
106.95	12257.2	$\binom{222}{2} = 24531$	0	1
220.75	49532.25	$\binom{444}{2} = 98346$	0	2
445.85	196192.85	$\binom{888}{2} = 393828$	0	3
893.35	788989.2	$\binom{1176}{2} = 1576200$	0	4
1777.75	3156223.9	$\binom{3552}{2} = 6306576$	0	5

Part 3

עלות חיפוש במערך עם היפוכים סמוכים אקראיים	עלות חיפוש במערך מסודר אקראית	עלות חיפוש במערך ממוין-הפוך	עלות חיפוש במערך ממוין	מספר סידורי i
228.8	2194.45	2694	221	1
580.75	5299.1	6272	443	2
1163.55	12341.7	14316	887	3
2324.95	28161.2	32180	1775	4
4663.95	63356.4	71460	3551	5
9326.75	140788.3	157124	7103	6
18657.85	310707.15	342660	14207	7
37291.45	677849.5	742148	28415	8
74533.9	1470105.55	1597956	56831	9
149167.9	3194236.2	3423236	113663	10

Part 4

I. We defined I as the number of inversions in the array, i.e., the total number of pairs of items of the array, where the item with the smaller index is greater than the item with the bigger index. That is,

I= $\sum_{i=0}^{n}\sum_{j=0}^{i}|\{\ list[i], list[j] \quad | \quad list[j] > list[i]\ \}|$.

And we define d_i as the number of items before the index i, that are greater than the item with index i. That means,

$d_i = \sum_{j=0}^{i}\{\ list[i], list[j] \quad | \quad list[j] > list[i]\ \}$ for each i.

Therefore, the total number of inversions, I, will be $I = \sum_{i=0}^{n} d_i$, as desired.

II. Let us remind the algorithm of finger insert. We start from the node holding the maximal key, and we go up until we find the first node that holds a smaller key than the key we are going to insert. Then we go up to its parent and come down to its left side until we find the correct place to add the new node.

Now, for the i-th insertion, the optimal case is when all items inserted beforehand are smaller than the i-th key, i.e., $d_i = 0$. And for each node holding a bigger key than i-th key we must go up (and then down) until we find the correct place of insertion. And let us remember that since we only go up and down, we may not visit all the bigger nodes, and only the bigger nodes on the path through the root. So, the total time for insertion in this case will be bounded by $O(\max(1, \log d_i))$.

So, the total time complexity for the sequence of such insertions will be $O(\log \prod_{i=0}^{n}(d_i + 2))$.

III. From AM-GM inequality we have:

$$\sqrt[n]{\prod_{i=0}^{n}(d_i + 2)} \leq \frac{\sum_{i=0}^{n}(d_i + 2)}{n}$$

Raise both sides to the power of n and we get:

$$\prod_{i=0}^{n} d_i \leq \left(\frac{\sum_{i=0}^{n}(d_i + 2)}{n}\right)^n$$

So, based on above inequality, and the fact that the function log is increasingly monotonous,

$$\log \prod_{i=0}^{n}(d_i + 2) \leq \log\left(\frac{\sum_{i=0}^{n}(d_i + 2)}{n}\right)^n = n\log\left(\frac{\sum_{i=0}^{n}(d_i + 2)}{n}\right)$$

And since $\sum_{i=0}^{n} d_i = I$, we get:

$$\log \prod_{i=0}^{n}(d_i + 2) \leq n\log\frac{I + 2n}{n} = n\log\left(\frac{I}{n} + 2\right)$$

Therefore, the time complexity will be $O(n\log(\frac{I}{n} + 2))$.

IV. As we can see in the following tables, the result of the experiment perfectly validates the theoretic upper bound we found above.

a. Sorted array:

i	Number of inversions (I)	Size of the array (n)	Upper bound $O(n\log(\frac{I}{n} + 2))$	Experiment result
1	0	222	222	228.8
2	0	444	444	580.75
3	0	888	888	1163.55
4	0	1776	1776	2324.95
5	0	3552	3552	4663.95

For all i, the result of the experiment is $O(n\log(\frac{I}{n} + 2))$.

b. Reversed sorted array:

i	Number of inversions (I)	Size of the array (n)	Upper bound $O(n \log(\frac{I}{n} + 2))$	Experiment result
1	24531	222	1512.65	2694
2	98346	444	3465.03	6272
3	393828	888	7813.76	14316
4	1576200	1776	17399.20	32180
5	6306576	3552	38346.09	71460

Similarly, all the result are within the bound (for c=2 in this case) of $O(n \log(\frac{I}{n} + 2)$.

c. Randomly sorted array:

i	Number of inversions (I)	Size of the array (n)	Upper bound $O(n \log(\frac{I}{n} + 2))$	Experiment result
1	12257.2	222	1296.09	2194.45
2	49532.25	444	3031.32	5299.1
3	196192.85	888	6926.84	12341.7
4	788989.2	1776	15631.83	28161.2
5	3156223.9	3552	34804.61	63356.4

Similarly, all the result are within the bound (for c=2 in this case) of $O(n \log(\frac{I}{n} + 2)$.

d. Sorted array with random inversion

i	Number of inversions (I)	Size of the array (n)	Upper bound $O(n \log(\frac{I}{n} + 2))$	Experiment result
1	106.95	222	291.12	228.8
2	220.75	444	586.21	580.75
3	445.85	888	1174.93	1163.55
4	893.35	1776	2350.82	2324.95
5	1777.75	3552	4696.49	4663.95

For all i, the result of the experiment is $O(n \log(\frac{I}{n} + 2))$.

Code Description

AVLNode Class:

This class describes a node in AVL tree. It contains a constructor, and a method called is_real_node.
The constructer produces a node, from given key and value.
Each node has a key, a value, a pointer to its right and left children, a pointer to its parent and its height as attributes.
The method is_real_node checks whether the node is real, i.e., it is not a virtual leaf (with height of -1). Time complexity is $O(1)$.
AVLTree Class:
This class describes an AVL tree. It contains a constructor, and the following methods:

- Search
- Finger_search
- Insert
- Perform_rotations
- Right_rotation
- Left_rotation
- Promote
- Finger_insert
- Delete
- Update_parent_reference
- Get_height
- Get_balance
- SuccessorCh
- Find_max
- Find_min
- Join
- Split
- Balancing
- Avl_to_array
- In_order_traversal
- Max_node
- Size
- Get_root

The constructor produces a new tree by producing its root (initially none) and giving it the size of 0.
Now, we will review the methods one by one, explaining what they do and how they work.

1. Search

The method takes a key and starts from the root, traverses the tree based on the key (left for smaller, right for larger)
Returns the node if found, or None if not, and path length. If the tree is empty (i.e., the root is none), it will return None and 0 (the length of the search path in number of edges)
Complexity is big O of the tree height $\boldsymbol{O(log\ n)}$.

2. Finger_search

Like search but starts from the maximum node instead of the root. Traverses up till the parent that the node is in its left subtree then traverses down as needed.

Worst cases are when we are searching node in the root's left subtree (smaller keys) so we have to traverse up till the root (log n) then down to the required one totally will cost maximum (2 log n).
Complexity is the tree height $\boldsymbol{O(log\ n)}$.

3. Insert:

Traverses the tree to find the correct position for the new node, same way as we traverse in search and we save this path in a list, path, each node we visited counts the number of the edges in the path (e) then inserts the node as a leaf and initialize virtual children.
Uses **def** promote to update heights and **def** perform_rotations to rebalance the tree if needed
Promote method returns how many nodes changed their height (h) we save if as p so in the end Insert will return (new node , e, p)
Complexity is $\boldsymbol{O(log\ n)}$ because to find the insertion mostly cost log n (the tree height) we use the helper methods that their time complexity is $\boldsymbol{O(log\ n)}$ so in the end the total cost will be at most $\boldsymbol{O}(3 * \boldsymbol{log\ n}) = \boldsymbol{O}(\mathbf{log}\,\boldsymbol{n})$.

4. Promote:

The method gets a node and traverses upward till the root, updates the height for the ancestors if needed and counts using the variable "p" the balanced ones after the updating till the first unbalanced one.
Complexity is $\boldsymbol{O(log\ n)}$ because we simply traverse from the node to the root, so the path length is the tree's height log n.

5. Perform _rotation:

The method gets new_node and its path such that the new_node is at the top. For each node we check the balance factor and rotate left or right as needed using **def** right_rotation and **def** left_rotation.
Complexity $\boldsymbol{O(log\ n)}$ because in the worst case, path length is logn (tree height) and time complexity for **def** right_rotation and **def** left_rotation is $\boldsymbol{O(1)}$.

6. Right _rotation:

Rebalances when the left subtree is too tall (balance factor > 1 or <-1) by making the left child L, the new root, then attaches L's right child LR as the left child of the old root, N, and make N the right child of L.
Updates parent pointers and heights of N and L.
Complexity $\boldsymbol{O(1)}$ since it involves a constant number of pointer updates.

7. Left _rotation:

The algorithm is symmetrical to the right rotation.
Complexity $\boldsymbol{O(1)}$, same as in right_rotation.

8. Finger_insert

We use the same algorithm as finger_search to find the correct place for the new node, we add to the path each node we visited, if we turned left we clear the path and continue appending, because the previous nodes are already balanced and we did not make any changes. Then we rebalance the nodes in the path and the ancestors of the subtree using **def** promote and **def** perform_rotations.
Complexity is $\boldsymbol{O(log\ n)}$ because to find the insertion place is similar to finger_search so path's length is at most 2(log n) and after it is built we use the helper methods that their time complexity is $\boldsymbol{O(log\ n)}$ so in the end the total cost will be at most $\boldsymbol{O}(3 * \boldsymbol{log\ n})$.

9. Delete:

First, we have 2 exceptional cases:

1) Tree has only one node and we want to delete it, so we simply convert root to none.
2) Tree has one child, and we want to delete the root so the root will be the child and its parent is none.

And now if we want to delete any other node, we have three cases:

1) The node has two children, then we replace the node with the successor.
2) The node has one child, so we replace it with its child.
3) The node is leaf, so we remove it.

To update the parents in cases 2 and 3 we used the helper method update_parent_reference
We build an array containing all the ancestors of the deleted node, we update their heights and rebalance them from the closest one till the root.
Before return, we update the tree size.
Complexity: since the method only traverses from the root down until the node we want to delete, the traversal path length will be at most the height of the tree. Also, all the helper methods are with complexities of $O(1)$ and are called at most $\log n$ times. So, the complexity of the delete method is $\boldsymbol{O(\log n)}$.

10. Update_parent_reference:

The method gets two nodes and updates their parents so that the new_node will replace the node that will be deleted.
Complexity : $\boldsymbol{O(1)}$.

11. Get_height :

Returns the node's height or -1 if the node is None.
Comlexity :$\boldsymbol{O}(\mathbf{1})$.

12. Get_balance : $\boldsymbol{O}(\mathbf{1})$:

Returns the node's height or -1 if the node is None.
Complexity: $\boldsymbol{O}(\mathbf{1})$.

13. SuccessorCh: $\boldsymbol{O}(\boldsymbol{log\ n})$

The method gets a node and returns its successor.
Complexity: The worst case we are in the root so its successor will be the leftest child in its right subtree, and the length of the traversal path until we get it will be the tree height $\boldsymbol{O}(\boldsymbol{log\ n})$.

14. Find_max:

The method gets a node, and returns the maximum node in its the subtree (the subtree that its root is node) by traversing down to the rightest node.
Complexity: since it traverses down to the rightest leaf, the complexity is big O of the length of the subtree whose root is the given node, so $\boldsymbol{O}(\mathbf{log}\,\boldsymbol{n})$**.**

15. Find_min:

The mrthod gets a node, and returns the minimum node in its the subtree (the subtree that its root is node) by traversing down to the leftest node.
Complexity: since it traverses down to the leftest leaf, the complexity is big O of the length of the subtree whose root is the given node, so $\boldsymbol{O}(\mathbf{log}\,\boldsymbol{n})$**.**

16. Join

The method gets a tree, a key, and a value as inputs. The key and the value make the joining node, that mean, firstly the method will make a new node using the give key and value.
Then, it takes care of the exceptional cases:

1. If the "self" tree is empty, it will simply take the second (given) tree and inserts the new node and marks it as "self".
2. If the given tree is empty, it will just insert the given node into the current tree.

Now, it will take care of our four general cases, based on what we learnt in the lecture.

1. The given tree is with greater keys than self tree and it is heigher than self tree.
2. The given tree is with greater keys than self tree and it is shorter than self tree.
3. The given tree is with smaller keys than self tree and it is heigher than self tree.
4. The given tree is with smaller keys than self tree and it is shorter than self tree.

We will provide description for the first case, and other three cases are symetric.
Since the given tree is with greater keys, it must be positioned on the right side of our tree. And since it is heigher than self tree, the method traverses from its root down left until it finds the first node with the same height as the height of self.root (or +1). Then it updates the poiners do that the new node will be in its place, the left child of it will be self.root and the right child of it will be the founded node. Then it will check for any unbalencedness from the added node upwards, and rebalances the tree if needed.

In the beging of each case, the method updates the tree size and in the end of each case it will invalidate the given tree, so that the user will not be able to call it anymore.
Complexity: since the traversal is from the heigher tree down to the point with same hight as the shorter tree, and then rebalancing is in the same path upwards, the complexity will be big O of the difference of height, or to be more general with a looser bound, we can say $\boldsymbol{O}(\mathbf{log}\,\boldsymbol{n})$.

17. Split

The method is given a node and splits the tree into two subtrees, one with greater keys than those of the given node, and one with smaller ones, and returns those subtrees as its output.
After making two empty trees, T1 and T2, by calling the constructor of the class, first, it locates the given node, by performing tree search based on its key. Then, it detaches the node, traverses up and joins the subtrees with larger keys to T2, and joins the subtrees with smaller keys to T1. Then, checks the balance of each tree and performs rebalancings if needed, using balancing method, and returns the trees.
Complexity: since we perform joins on all the mentioned subtrees, and each subtree is produced throughout traversing upward until the root, at most $O(\log n)$ joins are called each costing $O(\log n)$ time, however, based on the claimed we proved at the lecture, the total complexity will be $\boldsymbol{O}(\mathbf{log}\,\boldsymbol{n})$.

18. Balancing

It takes a node as input, checks its balance factor (the difference of height between its left child and its right child), and if it is not balanced, it will call the relevant rotation method.
Complexity: $\boldsymbol{O}(\mathbf{1})$ since the rotation methods are also with complexity of $O(1)$ and they are called only once.

19. AVL_to_array

Returns a sorted array of the elements of the tree, sorted by the node keys, by calling the helper method in_order_traversal, from the root down.
Complexity: is the same as the comlexity of helper function, called on the root of the tree, i.e., $\boldsymbol{O}(\boldsymbol{n})$, where n is the size of self tree.

20. In_order_traversal

A recursive method that performs an in-order traversal through the tree, starting from the root (in our case), by calling itself recursively first with the left node as input, adding the result into the output list, then adding its own info into the list, and then the same with the right node as input.
Complexity: since each node of the tree with the given node as root is visited, the time complexity is $\boldsymbol{O}(\boldsymbol{n})$ where n is the number of items in the tree whose root is the given node.

21. Max_node

Returns the node with the maximum key in the tree, simply by traversing from the root down right until the righternmost node, which holds the maximum key.
Complexity: since the method traverses all the way from the root to a leaf, the complexity is big O of the height of the tree, i.e., $\boldsymbol{O}(\mathbf{log}\,\boldsymbol{n})$.

22. Size

The method only returns the sizeT attribute of the tree, hence the time complexity: $\boldsymbol{O}(\mathbf{1})$.

23. Get_root

The method only returns the root attribute of the tree, hence the time complexity: $\boldsymbol{O}(\mathbf{1})$.